From Pixels to Language — Milestone 1 Discussion Guide (v2, final)

Everything we did, why we did it, what we found, and how to defend it

Contents

How to use this guide 2

1. The project in one page 2

2. How we worked: the AI-usage workflow applied 3
- 2.1 Think . . . 3
- 2.2 Plan . . . 3
- 2.3 Challenge with AI → Refine (where the plan changed, and why) . . . 3
- 2.4 Define (interfaces we fixed before building) . . . 4
- 2.5 Build incrementally . . . 4
- 2.6 Review . . . 4
- 2.7 Validate . . . 4

3. The journey of one image through the final model 5

4. What the data told us, and what we did with it 5

5. Models 6

6. Training mechanics, explained simply 6

7. Milestone-1 baseline results 7

8. The improvements, one change at a time 8

9. Final runs and how the final model was chosen 9

10. Grad-CAM: what the models look at 10

11. Benchmarking and what is in the report 10

12. Code map: what to read, in order (~2 hours) 11

13. Infrastructure 12

14. Glossary 12

15. Likely discussion questions, with short answers 14

16. Numbers cheat-sheet 15

How to use this guide

The discussion is **35% of your grade** and **individual**: you will be asked to explain *any* part of the project and to justify the decisions. This guide is organised so you can:

1. Get the big picture in 1 page (Section 1).
2. Understand *how we worked* using the course's AI-usage workflow (Section 2).
3. Follow one image through the whole pipeline (Section 3); if you can explain this, you understand the code.
4. Learn every result and what it means (Sections 4–10).
5. Know what is in the report and what is still missing (Section 11).
6. Practise with ~30 likely questions (Section 15) and a numbers cheat-sheet (Section 16).

The course says *"you steer the AI; you must understand and review generated code."* So after reading this, open the files listed in Section 12 and check that each explanation matches the code. Then practise explaining two W&B runs out loud.

1. The project in one page

Goal of the whole project. One visual encoder, three tasks on CUB-200-2011 (11,788 bird photos, 200 species, 10 captions each):

Task	Milestone	Role of the encoder
Classification (200 species)	**M1, now**	*trains* the encoder to see birds
Colorization (gray → colour)	M2 (18 Dec)	encoder = U-Net encoder, its 4 feature maps = skip connections
Captioning (image → text)	M2	frozen encoder, its last 7×7 / 12×12 grid feeds a text decoder

What M1 requires, and where it is:

Requirement (PDF)	Done	Where
EDA, cleaning, captions split/cleaned, augmentation strategy	✓	`src/data/*`, report §2
CNN from scratch	✓	`ScratchCNN`, report §4, §10
Transfer learning + comparison	✓	ResNet-50, EfficientNet-B0, ConvNeXt-T
Hyper-parameter tuning: grid **and** random, compared	✓	`src/training/tuning.py`, report §5, §10.7
Experiment tracking	✓	W&B, 53 runs
Grad-CAM, correct and wrong predictions	✓	report §8, §10.10
≥2 metrics, explained	✓	accuracy, top-5, macro P/R/F1, balanced acc., confusion
Benchmarking against published work	✓	report §9
CLI for training and inference	✓	`python -m src.cli ...`
Modular code, README, requirements	✓	repo

Headline results.

Model	Test accuracy
Scratch CNN, first version	52.5%
Scratch CNN, final (crop + cosine loss + Mixup/CutMix, 250 epochs)	**66.8%** (67.5% with TTA)
ConvNeXt-Tiny, Milestone-1 baseline	84.6%
ConvNeXt-Tiny, final (selected)	**87.0%** (**87.3%** with TTA)

There is no required accuracy in the PDF. What is graded is a complete, correct, honest and well-justified project.

2. How we worked: the AI-usage workflow applied

The course workflow is **Think → Plan → Challenge with AI → Refine → Define → Build incrementally → Review → Validate**. Here is what each step looked like in *this* project. The real problems and changes of direction are the most interesting part for the examiner, so learn them.

2.1 Think

- Break the task down: M1 = data understanding + an encoder trained by classification + all the required analyses. The encoder must be reusable in M2, so its *interface* matters from day one.
- Key insight: CUB is **fine-grained with ~27 training images per class**. Overfitting and small visual details will dominate every decision.

2.2 Plan

- Pipeline: `prepare → eda → train → tune → evaluate/gradcam → compare`, all through one CLI.
- Experiment plan: scratch baseline, linear probe, fine-tuning of three architectures, grid vs random tuning.
- Compute plan: we had a 6 GB laptop GPU, an 8 GB GPU and free Kaggle T4s. We chose Kaggle, with two machines × two GPUs running in parallel, and W&B so all runs land in one dashboard.

2.3 Challenge with AI → Refine (where the plan changed, and why)

These are real iterations. Each one is a good answer to *"what went wrong / what did you change?"*

1. **Duplicates and leaks.** The EDA found the same photo in both validation and test (a leak), and one photo labelled as two species. *Refinement:* pHash deduplication in `prepare`. Train/val copies are dropped, and the official test set is never touched.
2. **Kaggle silently ran on CPU.** The first jobs got no GPU and no internet (the account was not phone-verified). *Refinement:* a `doctor` command, and the trainer now **refuses to train on CPU** unless explicitly allowed. This caught the problem in seconds instead of wasting hours.
3. **Tuning v1 did not transfer.** Random search beat grid search, but the tuned settings gave **no gain** at 30 epochs (82.40% vs 82.38%). Diagnosis: 8-epoch trials reward settings that learn *fast*, not ones that end *best*. *Refinement:* **tuning v2** with 15-epoch trials, on the final model family, over 4 hyper-parameters.
4. **A teammate's suggestions.** A friend who did a similar project suggested (a) making the image "only the bird" and (b) trying the **cosine loss** instead of cross-entropy. We checked both against the data and the literature:
   - The EDA showed the bird is only **32% of the image** (median), which justifies cropping.
   - Barz & Denzler (WACV 2020) designed the cosine loss for small datasets trained from scratch, and report 51.9% → 68.0% on CUB from scratch.
   - Both were implemented and tested one at a time.

5. **The RandAugment result confirmed a design decision.** We had decided early on *not* to change colours in augmentation (colour identifies the species). RandAugment, which includes colour operations, turned out to be the *worst* augmented setting. That is evidence for the decision, not just a hunch.
6. **The Grad-CAM bug.** The first Grad-CAM grid assumed every model had the same input size; the final model uses 384 px. *Refinement:* the heat-maps are resized to the displayed image.

2.4 Define (interfaces we fixed before building)

Part	Contract
Config	`DEFAULTS` ← YAML (← `extends:` parent) ← `--set key=value`; saved in every run folder and checkpoint
Dataset item	`(image [3,H,W], label int, row index int)`
Encoder (`BaseEncoder`)	`forward_features(x) → 4 feature maps` (strides 4/8/16/32), `out_channels`, `gradcam_layer`, `is_pretrained`
Classifier	`forward(x) → logits [B,200]`, `param_groups()` (different LR for head and backbone)
Loss	`criterion(logits, target)`: target is an int label or soft labels (Mixup/CutMix)
Tracker	`log_metrics / log_summary / log_image / finish`; W&B and local files behind one interface

2.5 Build incrementally

The git history follows the pipeline order. Each piece was smoke-tested on CPU (1–2 training steps) before the next one:

- skeleton → data → EDA → models → training/tracking/tuning → evaluation/Grad-CAM/CLI;
- then the CPU guard, the results, bounding boxes, new augmentations, cosine loss + Mixup/CutMix, TTA, and the experiment configs.

A teammate (Hassan) added tests, folder inference and evaluation docs through **Pull Request #1**. It was reviewed, its 11 tests were run, and it was merged with a merge commit so his commits keep his name.

2.6 Review

- Every figure was looked at. That is how we caught that *strong* augmentation can remove the bird, and that pHash pairs at distance 3–4 are false positives.
- Every new option was smoke-tested before any GPU time was spent.
- The PR review: we read the code, ran the tests, and tried the new feature on a real checkpoint.

2.7 Validate

- The test set is used **only** for reporting. All model selection uses validation.
- A fresh clone of the repo was run end to end.
- Results were benchmarked on the **same test split and metric** as published papers.
- We stated the **noise level**: 598 val images give about ±1.4 points; 5,794 test images give about ±0.45.

3. The journey of one image through the final model

Follow one training photo, e.g. a Cardinal, 500×375 pixels.

1. **Download & cache (`prepare`)**: the HF record gives the image, label (e.g. 16 = Cardinal), description and file name. It is stored once as JPEG (high-res cache: shorter side up to 512, never upsampled, so it stays 500×375). `metadata.csv` gets split (train/val/test), the 10 cleaned captions, its bounding box and duplicate flags.
2. **Dataset (`CubClassificationDataset.__getitem__`)**: open the JPEG → RGB. (In crop runs: `crop_to_bird` cuts a square box around the bird with a 15% margin; the final model does **not** crop.)
3. **Augmentation (`build_train_transform("standard", 384)`)**:
   - `RandomResizedCrop(384)` takes 35–100% of the area at a random aspect and resizes it to 384×384;
   - horizontal flip with p = 0.5; rotation ±15°; brightness/contrast ±20%, saturation ±10%, **hue 0**;
   - `ToTensor` gives a float tensor **[3, 384, 384]** in [0,1];
   - `Normalize` with ImageNet mean/std (the statistics the backbone was pretrained with);
   - `RandomErasing` (p = 0.25) blanks a small rectangle to simulate occlusion.
4. **Batching**: 32 images → **[32, 3, 384, 384]**; labels **[32]**.
5. **Mixup/CutMix (80% of batches)**: pick Mixup (blend two images, λ ~ Beta(0.2, 0.2)) or CutMix (paste a rectangle). Labels become **soft** vectors **[32, 200]**, e.g. 0.7 Cardinal + 0.3 Summer Tanager.
6. **Encoder (ConvNeXt-Tiny)**: stem (4×4 conv, stride 4) → stage 1 **[32, 96, 96, 96]** → stage 2 **[32, 192, 48, 48]** → stage 3 **[32, 384, 24, 24]** → stage 4 **[32, 768, 12, 12]**. These 4 maps are what `forward_features` returns. Milestone 2 reuses them.
7. **Head**: global average pooling → **[32, 768]** → dropout 0.2 → linear 768→200 → **logits [32, 200]**.
8. **Loss**: cross-entropy with the soft targets and label smoothing 0.1. (Cosine variant: normalise the output vector, maximise its cosine with the target vector, + 0.1 × cross-entropy.)
9. **Backward + update**: mixed precision (FP16) with a gradient scaler; AdamW with **two learning rates**: head 1.25e-3, backbone 0.25 × 1.25e-3 ≈ 3.1e-4. The LR follows warm-up (1 epoch) then a cosine decay to 0.
10. **End of epoch**: evaluate on validation (eval transform: resize to 439 → centre crop 384, no augmentation). If val accuracy improved, save `best.pt`. Log everything to W&B.
11. **After training**: reload `best.pt` and evaluate on **test**, plain and with **TTA** (average the softmax of the image and its mirror). Write metrics, confusion matrix, most-confused pairs, per-class accuracy and curves.

4. What the data told us, and what we did with it

EDA finding	Decision
No validation split in CUB	stratified 10% of train (3/class, seed 42); tuning never touches test
Test imbalanced: 11–30 images/class	report **macro-F1** and balanced accuracy
Only ~27 train images/class	transfer learning; regularisation (label smoothing, WD, dropout, augmentation, Mixup/CutMix)
8 true duplicate pairs (2 val↔test leaks, 1 cross-species)	drop train/val copies; never modify test
8 grayscale images	keep for classification, flag for M2 colorization
Colour words in 98.7% of captions	**no hue jitter / no grayscale augmentation**
Bird covers **32% of the image** (median, min 5%; 2,739 images < 20%)	crop-to-bird experiments; motivates attention/zoom
Fine details (wing bars, bill colour)	higher resolution (384 px)

EDA finding	Decision
1,700 corrupted characters (U+FFFD), 1,306 duplicate captions, templates ×971	caption cleaning; template issue noted for M2
679 rare misspellings (0.09% of tokens)	no spell-checker (it breaks bird words); vocabulary cut-off (min freq 5 → 1,243 words, 99.5% coverage)



Figure 1: Bird size and the crop preprocessing

5. Models

- **Scratch CNN (ours, 11.3M parameters)**: a ResNet-18-style network.
  - Stem (3×3 conv stride 2, 3×3 conv, max-pool), then 4 stages × 2 residual blocks with 64→128→256→512 channels.
  - Kaiming initialisation, and the last BN of each residual branch starts at **zero**, so every block starts as an identity mapping. This makes early training stable.
- **No-skip ablation**: the same network without residual connections reaches only 44% *train* accuracy. It is an **optimisation** problem (the degradation problem), not overfitting.
- **Transfer**: ResNet-50 (23.9M), EfficientNet-B0 (4.3M), ConvNeXt-Tiny (28.0M), all ImageNet-pretrained.
  - **Linear probe**: freeze the backbone and train only the last layer, 67.6%.
  - **Fine-tuning** with discriminative LRs, 82.4% (ResNet-50), +14.8 points.
- **Why ConvNeXt wins**: it is a modernised ResNet (7×7 depthwise convolutions, LayerNorm, GELU, fewer activations), stronger on ImageNet, and better ImageNet models transfer better (Kornblith et al. 2019).

6. Training mechanics, explained simply

Technique	What it does	Why here
AdamW	Adam with weight decay applied directly to the weights	robust default for fine-tuning
Warm-up + cosine LR	small LR at first, then a smooth decay to 0	protects pretrained weights from the random head's early gradients; precise steps at the end
Discriminative LR	backbone LR = multiplier × head LR	the backbone is already good; adapt it gently. Tuning v2 found **0.25** better than 0.1
Label smoothing 0.1	target 0.9 for the true class, the rest spread over the others	prevents over-confidence with 27 examples/class
Mixed precision	FP16 math on tensor cores	about 2× speed, half the memory
Early stopping	stop after *patience* epochs without val improvement; keep the best checkpoint	avoids over-training; selection on val
Mixup / CutMix	train on mixed images with mixed labels	strong regulariser; the model must use partial evidence
TTA	average the predictions of the image and its mirror	free +0.6 points on average

7. Milestone-1 baseline results

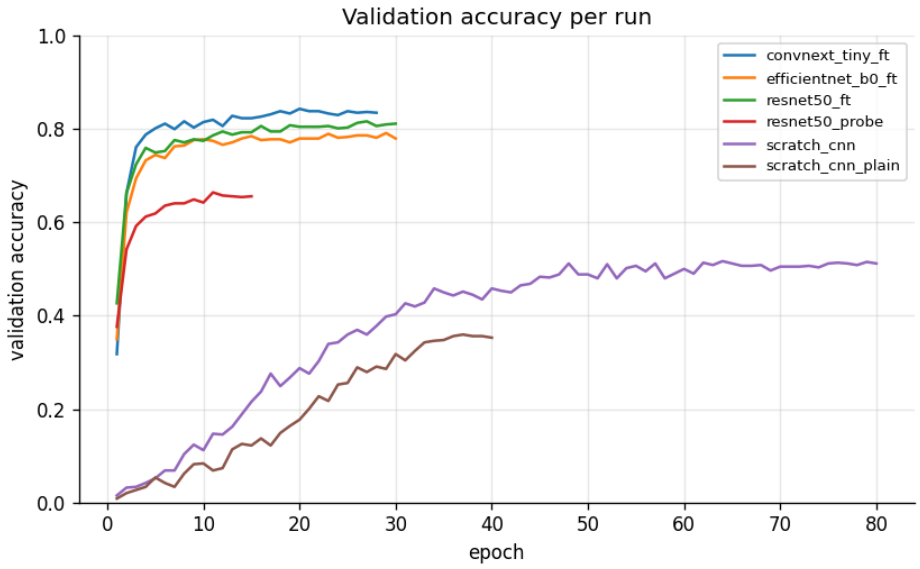


Figure 2: Validation accuracy per run (baseline)

Model	Test acc.	Key point
Scratch, no skips	36.5%	can't optimise (44% train acc.)
Scratch CNN	52.5%	overfits: 96% train vs 51% val
ResNet-50 linear probe	67.6%	frozen ImageNet features already informative
EfficientNet-B0	79.8%	4.3M params, very efficient

Model	Test acc.	Key point
ResNet-50 fine-tune (default / tuned v1)	82.4% / 82.4%	tuning v1 didn't help
ConvNeXt-T fine-tune	84.6%	best baseline

Tuning v1 (ResNet-50, 9 trials each, 8 epochs): random 80.4% vs grid 79.3% val. The learning rate dominates (row means 75.6 / 78.0 / 78.9% for LR 3e-4 / 1e-3 / 3e-3); weight decay changes accuracy by less than 0.7 points. Random search tests 9 different LR values instead of 3, so it explores the important dimension better (Bergstra & Bengio 2012).

8. The improvements, one change at a time

Rule: change **one thing**, compare with a re-run baseline, decide on **validation**.

Bird crop. +6.3 points for the scratch CNN (52.5 → 58.7) and +1.3 for ConvNeXt (84.6 → 85.9). It removes the background (shortcuts like the red feeder) and acts as a ×3 zoom on the bird. *Caveat:* it uses **ground-truth boxes at test time**, an annotation a real user would not give, so these results are labelled separately.

Resolution 384 px. Full image: same test accuracy (84.6%). With the crop: +0.3. The crop already zooms in, so the two partly do the same thing, and 384 px costs 2.3× more time per epoch.

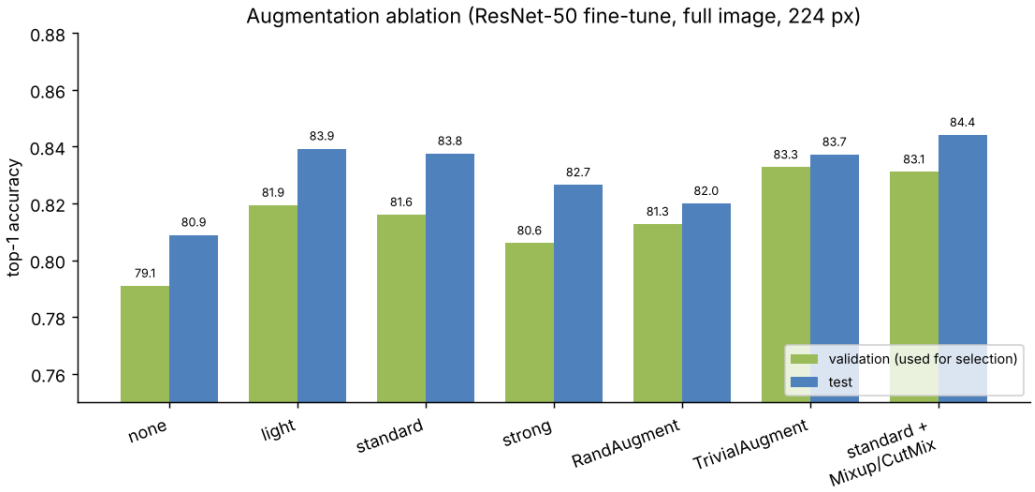


Figure 3: Augmentation ablation

Augmentation ablation (ResNet-50). * none 80.9% → light 83.9% ≈ standard 83.8%, so geometric augmentation gives +3 points. * strong 82.7%: it can crop the bird out. * **RandAugment 82.0%: worst**, because its colour operations change species colours. This confirms our design decision. * TrivialAugment 83.7%. * **standard + Mixup/CutMix 84.4%: best**.

Cosine loss vs cross-entropy. Cross-entropy can be lowered just by *inflating* the logits, which is how it overfits small datasets. The cosine loss looks only at the *direction* of the output vector. * Scratch CNN: +1.0 test / +2.3 val. This matches the paper's claim. * ConvNeXt: +1.5 val but only +0.1 test. * ResNet-50: −0.8.

So it helps from scratch; with pretraining the effect is inconsistent.

Mixup/CutMix. +0.65 for ResNet-50 and +0.56 for ConvNeXt with crop (86.5%). With 150 epochs it was the biggest gain for the scratch CNN (59.7 → 66.0). Note: the "train accuracy" of mixed runs (~67%) is measured on mixed images, so it is not comparable with validation.

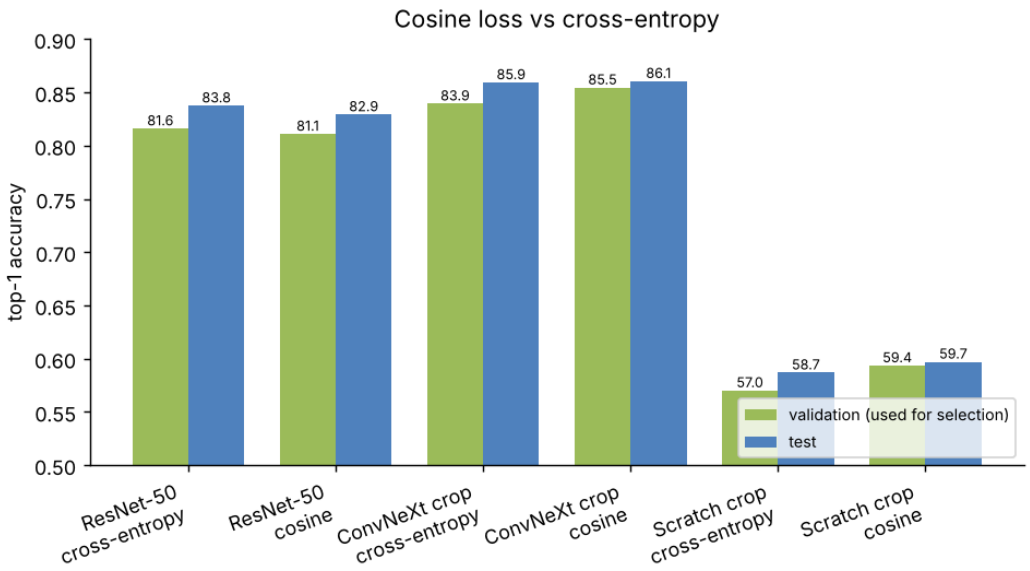


Figure 4: Cosine vs cross-entropy

TTA. Improved **all 16 runs**, +0.58 points on average (+0.22 to +1.07).

Tuning v2 (ConvNeXt + crop, 8 random trials × 15 epochs, 4 hyper-parameters). Best val 84.5% after only 15 epochs. Accuracy correlates with the **backbone LR multiplier** (r = +0.63) and the head LR (r = +0.63), and weakly negatively with weight decay. Lesson: for fine-grained birds the backbone must adapt *more* (0.25 instead of 0.1).

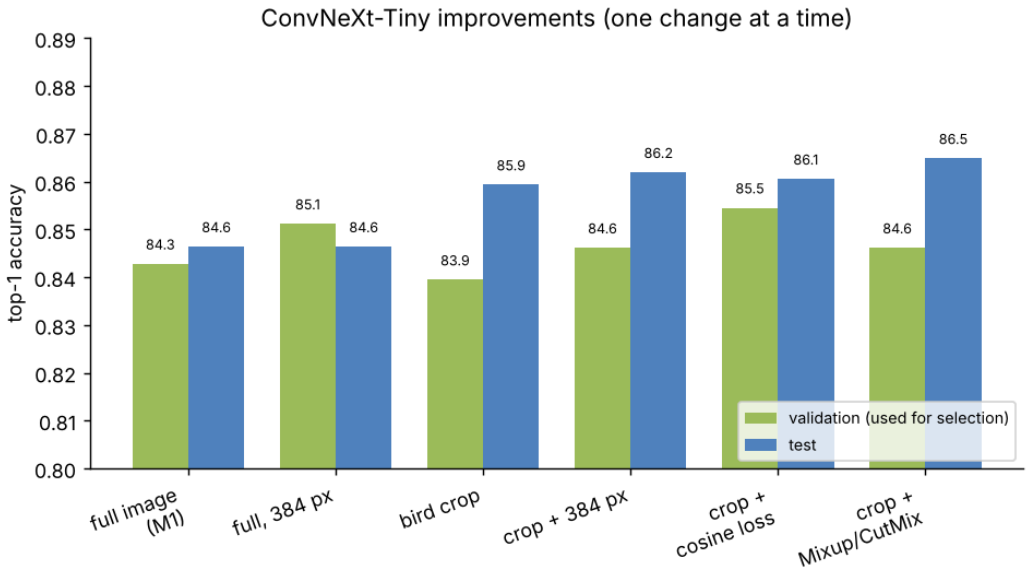


Figure 5: ConvNeXt: one change at a time

9. Final runs and how the final model was chosen

Three ConvNeXt variants combining everything that helped (384 px, Mixup/CutMix, tuning-v2 settings, 40 epochs), plus a 250-epoch scratch CNN:

Run	Crop	Loss	**Val**	Test	Test + TTA
ConvNeXt-T, full image (selected)	none	CE	**87.3**	87.0	87.3
ConvNeXt-T, bird crop	GT box	CE	86.3	87.3	87.5
ConvNeXt-T, bird crop, cosine	GT box	cosine	86.8	86.9	87.2
Scratch CNN, all improvements, 250 ep.	GT box	cosine	69.1	66.8	67.5

Selection = highest validation accuracy. That is the **full-image** model, which needs **no bounding box**, so it is also the fair, deployable one. The crop model has a slightly higher *test* score (+0.3), but choosing it for that reason would mean selecting on the test set, and the difference is within the ±0.45 noise anyway. **Lesson:** once the model is strong and sees the image at 384 px, cropping adds little.

We did **not** retrain on train + val (the option exists): without a held-out set we could not verify the result, and +11% data would likely change accuracy by less than the noise.

10. Grad-CAM: what the models look at

How it works. Take the last conv stage's activations A^k. Weight each channel k by the average gradient of the class score: $\alpha_k = \text{mean}_{ij}\, \partial y^c / \partial A^k_{ij}$. The map is ReLU$(\sum_k \alpha_k A^k)$, upsampled to image size. Bright pixels are the ones that increase the class score.

Findings.

1. Baseline: the pretrained models look at the bird. The scratch CNN looked at the **red feeder** instead of the hummingbird (a background shortcut), which motivated the crop.
2. Final vs M1 ConvNeXt: the final maps are **tighter, on head and breast**, and the model is **less over-confident** on its mistakes (e.g. 0.94 → 0.64).
3. Several of the final model's most confident "mistakes" are probably **label errors**. The image labelled *Green Kingfisher* shows a black-and-white **Pied** Kingfisher, which is what the model predicted. Experts estimated that ≥4% of CUB labels are wrong (Van Horn et al., CVPR 2015).
4. Shared hard cases: crow vs raven (size and bill), terns, gulls, *Empidonax* flycatchers, and the two nightjars (also the pair with a duplicated, double-labelled photo).
5. The final scratch CNN still attends next to the bird, and its confidences saturate at 1.00 (cosine scores are not calibrated probabilities).

11. Benchmarking and what is in the report

Benchmarking (same official test split, top-1 accuracy):

Method	Input	Top-1
ResNet-50 baselines (published)	448 px	84.5–85.3%
NTS-Net (part-based, published)	448 px	87.5%

Method	Input	Top-1
TransFG (ViT-B/16, ImageNet-21k, published)	448 px	91.7%
Ours, final ConvNeXt-T (no annotations)	384 px	**87.0% (87.3% TTA)**
ResNet-50 from scratch, CE / cosine (published)	448 px	51.9% / 68.0%
Ours, scratch CNN baseline / final (GT boxes)	224 px	52.5% / 66.8%

Fair-comparison points: same split and metric; differences in resolution, pretraining data and epochs explain most of the gaps; we report a single seed; the crop rows use ground-truth boxes.

Report map (33 pages):

§	Content
1	Introduction, dataset, summary
2	Data: splits, class distribution, sizes and quality, grayscale, duplicates, captions, augmentation, EDA → modelling
3	Models (scratch, ablation, transfer strategies, encoder interface)
4	Baseline experiments: scratch vs transfer, residual ablation, architectures
5	Tuning v1 (grid vs random) and why it didn't transfer
6	Experiment tracking (**3 W&B screenshots missing**)
7	Evaluation: metrics, interpretation, confused pairs
8	Grad-CAM (baseline)
9	Benchmarking
10	**Improving the models**: crop, resolution, augmentation ablation, cosine loss, scratch, TTA, tuning v2, final runs, final Grad-CAM
11	Final selection
12	Failures and limitations
13	Reproducibility (exact commands)
14	Team contributions

Still missing before submission: 1. team number; 2. names and IDs of the 3 other members; 3. 3 W&B screenshots (runs table, overlaid curves, tuning parallel-coordinates); 4. the Google Drive link with the checkpoints (`best.pt` of the final runs, from the Kaggle Output tabs); 5. the final zip `TeamXX.zip`.

12. Code map: what to read, in order (~2 hours)

File	What to understand
`src/cli.py`	every command and its arguments
`src/config.py`	defaults, YAML `extends`, `--set` overrides
`configs/improve/final_full_ce.yaml`	the final model's exact settings

File	What to understand
`src/data/prepare.py`	download, cache, val split, bounding boxes, pHash dedup
`src/data/datasets.py`	`crop_to_bird`, dataset item, dataloaders, train+val option
`src/data/transforms.py`	augmentation levels, why no hue
`src/data/text.py`	caption cleaning, vocabulary
`src/models/encoders.py`	`BaseEncoder`, `ScratchCNN`, registry, torchvision wrappers
`src/models/classifier.py`	head, cosine head, discriminative LR groups
`src/training/trainer.py`	the training loop, Mixup/CutMix, AMP, early stopping
`src/training/losses.py`	cross-entropy vs cosine loss
`src/training/tuning.py`	grid / random / optuna, comparison plots
`src/evaluation/metrics.py`	metrics, TTA
`src/visualization/gradcam.py`	hooks, Grad-CAM math
`src/pipelines/classification.py`	how everything is glued (incl. Hassan's `predict_folder`)
`tests/`	Hassan's tests (metrics, Grad-CAM)

13. Infrastructure

- **Kaggle**: free T4 GPUs. Each machine has **2× T4**; our runner gives each GPU its own queue, so 2 machines = 4 experiments in parallel. Code is cloned from GitHub inside the job; data is prepared there.
- **W&B** (college account, project `cub-pixels-to-language`): every run's config, curves, test metrics and checkpoint path. Filter by tags: `improve`, `final`, `tuning`, `ablation`.
- **GitHub**: full history, your commits plus Hassan's PR. Result tables and figures are in `outputs/results/`, report sources in `report/`.

14. Glossary

- **Fine-grained classification**: distinguishing sub-categories (species) of one category (birds).
- **Transfer learning / fine-tuning / linear probe**: reuse pretrained weights; train all layers / only the last layer.
- **Overfitting**: train ≫ val accuracy; the model memorises. **Underfitting**: it can't even fit the training data.
- **Residual (skip) connection**: output = F(x) + x; gives gradients a short path.
- **BatchNorm / LayerNorm**: normalise activations to stabilise training (ConvNeXt uses LayerNorm).
- **Label smoothing**: soften one-hot targets.
- **Mixup / CutMix**: blend / paste two images, mix their labels.
- **Cosine loss**: $1 - \cos(\text{normalised output, one-hot target})$.
- **TTA**: average predictions over augmented copies at test time.
- **Macro-F1**: per-class F1 averaged with equal class weights.
- **Top-5 accuracy**: the true class is among the 5 most likely.
- **Grad-CAM**: gradient-weighted activation map showing where the evidence for a class is.
- **pHash**: perceptual image hash; similar images give hashes with a small Hamming distance.
- **Proxy (in tuning)**: a short, cheap training run used to rank hyper-parameters.

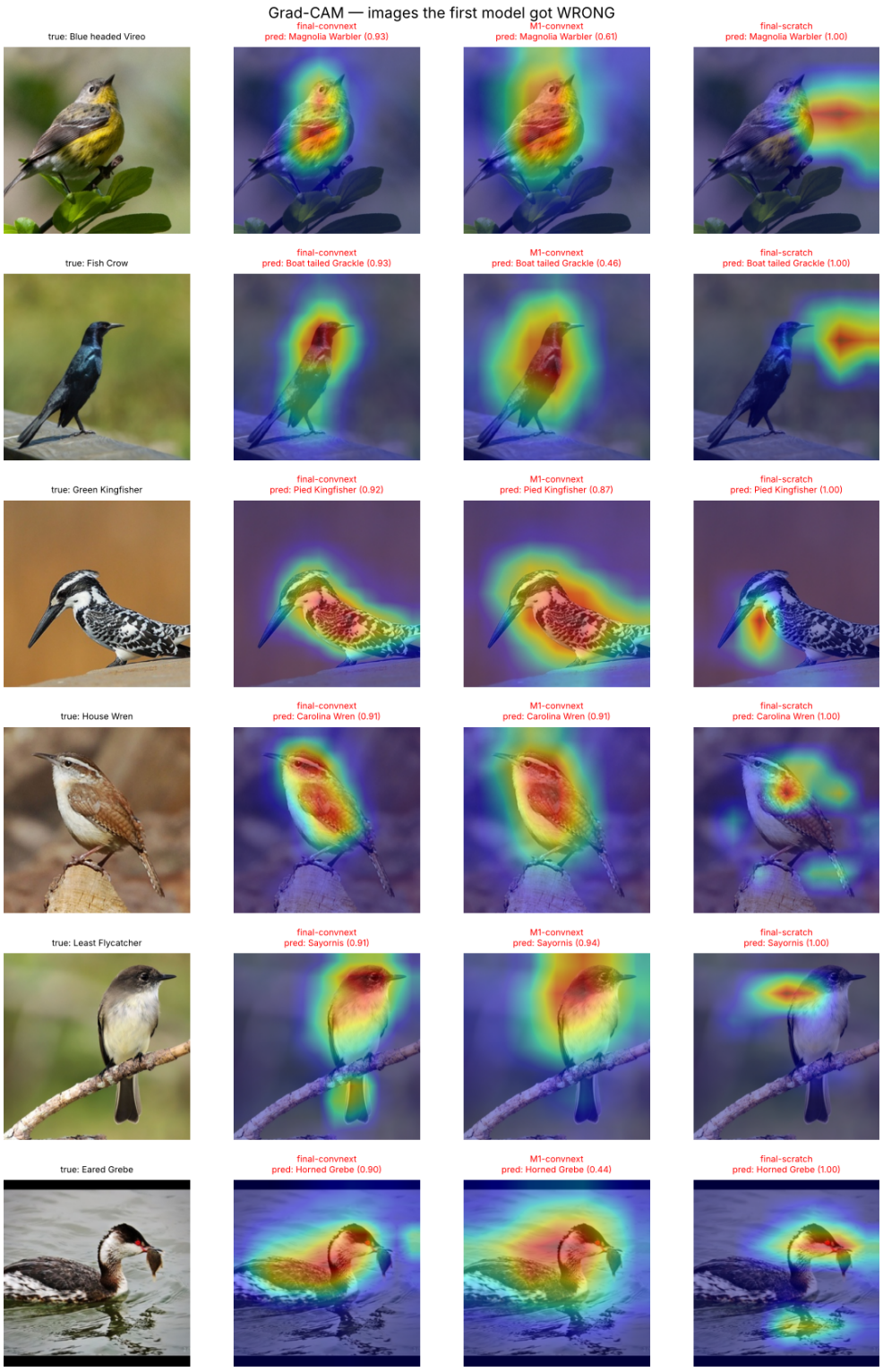


Figure 6: Final ConvNeXt's most confident mistakes vs M1 ConvNeXt and the final scratch CNN

15. Likely discussion questions, with short answers

1. **What is the project, and what does M1 deliver?** One encoder for classification, colorization and captioning. M1 trains it via classification, with EDA, tuning, tracking, Grad-CAM and benchmarking.
2. **Why a validation split?** To select models without looking at the test set. Selecting on test gives optimistic scores.
3. **Why not modify the test set when you found duplicates?** To stay comparable with published results on the same 5,794 images.
4. **How did you find duplicates?** With perceptual hashing; distance ≤ 2 was verified visually as the same photo.
5. **Why macro-F1?** The test set is imbalanced (11–30 images per class); macro gives every species equal weight.
6. **Why no hue augmentation?** Colour is a species feature. Hue changes create wrongly-labelled images. RandAugment (which has colour ops) was the worst setting, which supports this.
7. **Why is transfer learning so much better than scratch?** 27 images per class can't teach generic features; ImageNet already did that with 1.2M images.
8. **Linear probe vs fine-tuning?** Frozen features separate "bird vs dog". Fine-tuning learns the fine-grained cues (+14.8).
9. **Why a smaller LR for the backbone, and why did 0.25× beat 0.1×?** To avoid destroying good features; but birds need the deep layers to adapt *enough*, and tuning v2 showed more adaptation helps.
10. **What does warm-up do?** It avoids large updates while the random head produces large gradients.
11. **Overfitting evidence?** The scratch CNN: 96% train vs 51% val, and the val loss stops improving while the train loss keeps falling.
12. **Why did the no-skip CNN fail?** Underfitting from optimisation difficulty (44% train). Skip connections fix gradient flow.
13. **Grid vs random search?** Same budget; random tests more distinct values of the important parameter (LR) and won.
14. **Why didn't tuning v1 help, and what did you change?** The 8-epoch proxy favoured fast learners. v2 uses 15 epochs, the final model family and 4 hyper-parameters.
15. **What does Mixup/CutMix do, and why is train accuracy low with it?** It mixes images and labels as regularisation; train accuracy is measured on mixed images, so it is not comparable.
16. **What is the cosine loss, and did it help?** Only the output's direction matters, so it can't overfit by inflating logits. It helped from scratch (+1.0), but was inconsistent for pretrained models.
17. **Why did cropping help, and is it fair?** It removes background and zooms ×3. It uses GT boxes at test time, so it is reported separately; the final model doesn't use them.
18. **Why is your final model the full-image one, even though the crop model scored higher on test?** Selection is on validation; choosing on test would be cheating. The difference is within noise anyway.
19. **What is TTA, and how much did it give?** Flip averaging: +0.58 on average, and all runs improved.
20. **Why 384 px and not 448?** GPU time. And the crop/zoom effect already captured much of the benefit.
21. **Explain Grad-CAM.** Gradients of the class score weight the last feature maps → ReLU → upsample.
22. **What did Grad-CAM reveal?** The scratch CNN used the feeder (a shortcut). The final model has tighter maps and is less over-confident. Some "errors" are label errors.
23. **How noisy are your numbers?** Val ±1.4 points, test ±0.45 (binomial standard error), single seed.
24. **How do you compare with published work?** Same split and metric. 87.0% at 384 px beats the 448 px ResNet-50 baselines and is close to NTS-Net; the gap to TransFG comes from ImageNet-21k, ViT and part selection.
25. **Why is top-5 (97%) so much higher than top-1 (87%)?** Most errors are between sibling

species.

26. **What are the most confused pairs, and why?** Terns, gulls, flycatchers, crow/raven, nightjars: differences in size and subtle marks, plus label noise.
27. **How is the code modular?** Encoder registry + `BaseEncoder`, Tracker interface, config-driven, CLI; the trainer doesn't know which model, dataset or tracker it gets.
28. **How do you guarantee reproducibility?** Fixed seed, a stored split, the config saved in every run and checkpoint, exact commands in the README, and a fresh-clone test.
29. **How will M2 use this?** The ConvNeXt `forward_features` maps become U-Net skips (colorization). The frozen encoder's grid feeds an attention LSTM/Transformer (captioning).
30. **What would you do with more time?** 448 px, ConvNeXt-S/B or ImageNet-21k weights, several seeds, a bird detector for annotation-free cropping, cleaning label noise, multi-fidelity tuning.

16. Numbers cheat-sheet

Fact	Value
Images / classes / captions per image	11,788 / 200 / 10
Train / val / test (after dedup)	5,392 / 598 / 5,794
Train images per class	26–27
Test images per class	11–30
Bird area (median)	32% of the image
Duplicates (true) / leaks / cross-species	8 / 2 / 1
Captions after cleaning	116,574
Vocabulary (min freq 5)	1,243 words (99.5% of tokens)
Scratch baseline → final	52.5% → 66.8% (67.5% TTA)
ResNet-50 probe / fine-tune	67.6% / 82.4%
ConvNeXt baseline → final	84.6% → **87.0% (87.3% TTA)**
Final top-5 / macro-F1	97.1% / 87.0%
Best augmentation / worst	standard + Mixup/CutMix 84.4% / RandAugment 82.0% (none 80.9%)
Tuning v1 best val: grid / random	79.3% / 80.4%
Tuning v2 best: LR / backbone mult. / WD / LS	1.25e-3 / 0.25 / 1.3e-5 / 0.11
TTA average gain	+0.58 points
Total runs in W&B	53